

Ledger Reconciliation Engine — Technical Approach & Pitch Video Script

Prepared for the Razorpay AI Buildathon — Track 4: AI Finance Controller

Part 1 — The Approach, Explained in Detail

The problem in one sentence

A merchant's bank account receives one netted credit per settlement batch — 80 to 140 individual payments minus gateway fees and 18% GST, landing T+2 — and the job is to mathematically prove which specific payments make up each credit, without any shared key linking the two sides.

Why this is hard

There is no `payment_id` on the bank statement. All you have is a date and an amount. So "reconciliation" is really a constraint-satisfaction problem: find the subset of unassigned payments whose fees-and-tax-adjusted net sums to exactly this bank credit — and do it for every credit, while refunds and chargebacks quietly inject negative adjustments into later batches, and floating-point rounding threatens to make $100.00 - 2.50$ evaluate to 97.499999 .

The core design principle: deterministic first, probabilistic second

Two rules govern every decision in the system: never fabricate a match (precision is prioritized over recall), and use plain algorithms wherever they work, reserving the LLM strictly for the one sub-problem that is structural rather than arithmetic. This is the single idea the whole architecture hangs off, and it is also, almost word for word, the "AI judgment" criterion the buildathon judges score on: justify where AI helped and where traditional software was the right tool.

The five-tier pipeline

The engine runs candidates through progressively more expensive tiers. Anything solved by an earlier tier is removed from the pool before the next tier runs, so the search space keeps shrinking.

- **T0 — Key-based enrichment (deterministic).** Joins `orders.csv` and `settlements.csv` on `payment_id`. Even a payment missing from the internal ledger is kept in the pool, because the bank still credited the merchant for it — the system is designed to expect missing data rather than assume clean joins.
- **T1 — Date-window arithmetic (deterministic).** Checks whether all unassigned payments on one date sum exactly to a bank credit. This rarely clears much on its own (late captures break date alignment) but it narrows the field for T2. All comparisons happen in integer paise (`int(round(v * 100))`) specifically to kill float drift before it can hide a real mismatch.
- **T2 — Constrained subset-sum via dynamic programming (deterministic).** This is the computational core. Rather than searching for which ~120 of 150 candidates sum to the target (a

combinatorially huge search), the engine computes the small excess — candidate pool total minus target — and searches for the few items to *exclude* to reach that excess instead. That turns a ~1.7M-iteration problem into roughly 10K iterations. The reachable-sum frontier is represented as a single Python big-integer bitmask (bit *s* set = sum *s* is reachable), so an entire DP relaxation step is one bitwise shift-and-OR at C speed instead of a Python loop over a dict. This rewrite took the full pipeline from 233.7 seconds to 1.72 seconds and — more importantly — let the search *complete* instead of hitting an iteration cap, lifting credits cleared from 30/83 to 55/83.

- **T3 — Break classification agent (probabilistic, LLM-assisted).** Credits that survive T0–T2 have a real, non-arithmetic reason for failing: a batch split across two bank credits, a duplicate UTR, a late settlement. The LLM is deliberately never asked to propose which payment IDs make up the match — that is exactly the subset-sum task T2's DP already solves exactly, and exactly the task LLMs are worst at. Instead, T3 sends the LLM diagnostics only — window deficit, excess in paise, candidate count, nearby credits, whether negative entries are present — and asks for a break classification plus a structured hypothesis (an action like `expand_window`, `merge_with_utr`, or `accept_with_tolerance`, with parameters and a confidence score). With no API key configured, T3 falls back to a deterministic refund-aware heuristic, so the full pipeline still runs end to end with zero LLM dependency.

- **T4 — The arithmetic verifier and uniqueness gate (deterministic).** T3's hypothesis is never trusted as stated — it is *tested*. T4 re-runs real arithmetic against the proposed action and rejects anything that misses the target by more than ₹0.50 or reuses an already-assigned entry. The deeper contribution here is the uniqueness gate: checking that a subset sums to the target is tautological (T2's solver only ever returns subsets that sum to the target — the check can never fail, so on its own it proves nothing). The real question is whether it is the *only* subset that sums to the target. T4 re-derives the candidate pool and counts distinct solutions with the same exclusion-DP shape as T2, but enumerating every path instead of stopping at the first. One solution → accept as proven (UNIQUE_MATCH, confidence 1.0). Two to ten solutions → accept only the entries common to every valid solution, flag the rest as ambiguous (AMBIGUOUS_PARTIAL, confidence 0.75). More than ten → reject outright and route to the exception queue as SUM_COLLISION. This deliberately lowers the auto-clear rate to raise precision — measured at 0.578 before the gate and 0.625 after it on the same claims, rising to 0.808 when the confidence threshold is tightened to 0.95.

What happens to what doesn't clear

Every credit the engine declines to auto-clear becomes a structured exception row, not a dataframe dump: a break code (WINDOW_DEFICIT, NO_CANDIDATES, SUM_COLLISION, UNRESOLVED, or T3's classification), a materiality tier (High >₹50k, Medium >₹5k, Low), age in days, the delta against the closest subset found, T3's hypothesis, a suggested next action, and JSON evidence naming the closest candidate entries — so a controller can see why it broke and what to do about it, not just that it broke.

From reconciliation to cash visibility

`build_cash_position()` turns the verified output into a forward-looking treasury view: value already verified and settled, value stuck at risk in unresolved credits, a 7-day expected-inflow forecast, a confidence interval bracketing "every at-risk credit is lost" against "every one is recovered," and cash-at-risk aged into under-3-day, 3-to-7-day, and over-7-day buckets. Because the uniqueness gate

refuses unevicenced matches, the "verified settled" figure this module reports is one a controller can actually rely on — a higher auto-clear number bought with coin-flip matches would be confidently wrong, which is the worst property a cash forecast can have.

Auditability

`export_audit_trail()` emits one row per payment: which credit it cleared into, which tier made the call, a reason code, a confidence score, JSON evidence, and an ISO-8601 timestamp — all tagged with a `run_id` so any assignment can be defended after the fact. This is the direct answer to the "traceability" and "explainability" language in the track brief.

Evaluation methodology

A synthetic data generator injects realistic break types (late settlements, refunds, chargebacks, missing orders) at a chosen difficulty and seed, and a custom harness (`eval/harness.py`) measures precision, recall, and auto-clear rate tier by tier, plus wall-clock throughput. The headline run — 5,538 payments across 83 credits, seed 42, medium difficulty — reconciles in 1.72 seconds at roughly 3,212 payments/second, with the ablation table showing exactly what each tier contributes and exactly what the uniqueness gate costs and buys back in precision.

Stack

Python, pandas, and numpy for the ledger and DP work; Groq (`openai/gpt-oss-20b` by default) for the two LLM touchpoints — T3's break classifier and a natural-language Q&A; agent that answers questions from a compact, pre-verified structured summary rather than raw rows, so it can never invent a number; Streamlit and Plotly for a six-page dashboard (Overview, Reconciliation, Analytics, Exception Queue, Cash Position, Ask); pytest for a 115-test suite (114 passing, 1 skipped without an API key).

Part 2 — Video Script

Two versions below: a full 5-minute script and a tightened 2.5-minute cut. Bracketed lines are screen-recording cues, not spoken text. Timings assume a natural, unhurried speaking pace (~140–150 words/minute) — read them aloud once before recording and trim to fit your own pace.

5-minute version

[0:00–0:25] Hook + problem — show the README problem statement or a settlement CSV on screen

"A merchant gets one bank credit that quietly represents 80 to 140 separate payments, minus fees and GST, landing two days late. There's no shared key connecting that credit back to the payments. Refunds and chargebacks get buried inside later credits as negative adjustments. Proving, mathematically, which payments make up which credit — that's the reconciliation problem, and it's one every payments business has to solve continuously."

[0:25–1:10] Approach overview — show the ARCHITECTURE.md tier diagram or a whiteboard-style slide of T0–T4

"I built a five-tier pipeline that gets progressively more expensive, and only escalates a credit to the next tier if the cheaper one couldn't solve it. Key-based joins first. Then date-window arithmetic. Then a

dynamic-programming subset-sum solver for the harder cases. Only after all of that fails does an LLM even get involved — and even then, only to classify the *shape* of the failure, never to do the arithmetic. Everything the LLM proposes gets independently re-verified by deterministic code before it's trusted. That's the core design rule: deterministic first, probabilistic second, and never fabricate a match."

[1:10–2:00] Live demo — screen-record the Streamlit dashboard: Overview page, then Reconciliation page

"Here's the dashboard a controller would actually use. [click through Overview] Auto-clear rate, precision, rupees verified, rupees at risk, throughput — all headline numbers. [click Reconciliation] Every payment's tier-by-tier journey, and a downloadable audit trail — one row per payment, which tier cleared it, what evidence backs the decision, timestamped and tied to a run ID. Nothing here is a black box."

[2:00–3:00] The key innovation + AI judgment — show the T4 uniqueness-gate table from ARCHITECTURE.md or the Analytics page

"The part I'm proudest of is in the verification tier. Finding a subset of payments that sums to the credit is easy — with 80 to 160 candidates, there are usually several subsets that sum to the same number. Picking one and calling it solved is a coin flip dressed up as arithmetic. So the verifier counts every distinct subset that sums to the target. Exactly one solution means the match is mathematically proven. Two to ten means only the payments common to every possible solution get cleared. More than ten, it's rejected outright and goes to a human queue. That one change dropped my auto-clear rate on purpose — and pushed precision from 0.58 to over 0.80. For a financial ledger, an honest exception beats an unevidenced clear, every time."

[3:00–4:00] Failure recovery — show a before/after, e.g. terminal timing output or the performance table

"Two things broke along the way, and both changed the design. First: my original subset-sum solver hit an iteration cap and silently gave up on solvable credits — it wasn't just slow, it was wrong. I rewrote the reachable-sum search as a single big-integer bitmask, so a whole DP step became one bitwise operation instead of a Python loop. That took the full pipeline from 233 seconds to 1.7 seconds, and because the search could now actually finish, it cleared 55 credits instead of 30 — a correctness fix disguised as a performance fix. Second: I originally asked the LLM to propose which exact payment IDs made up a match. It was bad at it — worse than the deterministic solver I already had. So I redesigned T3 to have the LLM classify the *type* of break instead, and let the DP keep owning every number."

[4:00–4:40] Results + rigor — show the ablation/metrics table

"On a synthetic benchmark of 5,538 payments across 83 credits with injected refunds, chargebacks, and late settlements: the engine reconciles in 1.7 seconds at over 3,000 payments a second, and the full evaluation harness — precision, recall, auto-clear rate, per-tier ablation — is reproducible with one command. A hundred and fourteen of a hundred and fifteen unit tests pass; the one skip only fires without an API key configured."

[4:40–5:00] Close

"This is reconciliation you can actually trust a cash forecast on, because every number it reports is one the system can prove, not just compute. Thanks for watching."

2.5-minute version

[0:00–0:20] Hook

"A merchant's bank credit represents 80 to 140 separate payments netted together, with no shared key back to the individual ledger. Proving exactly which payments make up which credit — with refunds and chargebacks hidden inside — is the problem I set out to solve."

[0:20–0:50] Approach

"I built a five-tier pipeline: key joins, then date arithmetic, then a dynamic-programming subset-sum solver, escalating only what the cheaper tier couldn't clear. An LLM only enters at the very end, and only to classify *why* something failed — never to do the math. Every AI suggestion gets independently re-verified by deterministic code before anything is trusted."

[0:50–1:35] The innovation + AI judgment

"The core contribution is the verification gate: finding a subset of payments that sums to a bank credit is easy, but with over a hundred candidates, several different subsets often sum to the same number. So instead of stopping at the first match, I count every subset that reaches the target. Exactly one means the match is mathematically proven; more than ten means it's rejected and routed to a human. That deliberately traded auto-clear rate for precision — and took precision from 0.58 to over 0.80. In finance, an honest exception beats a fabricated match."

[1:35–2:05] Failure recovery

"Two failures shaped this. My first subset-sum solver hit an iteration cap and silently abandoned solvable credits — I rewrote it as a single bitmask operation, cutting runtime from 233 seconds to 1.7 seconds and, more importantly, letting it actually finish and clear correctly. And my first LLM design asked it to propose the matching payments directly — it was worse at that than my own solver, so I redesigned it to classify break types only, and kept every number deterministic."

[2:05–2:30] Results + close

"On 5,538 payments and 83 credits with injected breaks, it reconciles in 1.7 seconds at 3,000-plus payments a second, with 114 of 115 tests passing and a full reproducible evaluation harness. This is reconciliation a finance team can actually build a cash forecast on. Thanks for watching."